Chapter 19

A Web Application Hacker's Toolkit:

Many web application attacks can be done using just a browser, but most need extra tools. These tools either work as browser extensions or run separately to help interact with the target application. One important tool is an intercepting proxy, which lets you see and change the data (HTTP messages) between your browser and the website. Over time, these proxies have become advanced toolkits with many features for testing web security. Another key tool is a web application scanner, which automates tasks like analysing the website and finding security weaknesses. These scanners are helpful but also have some limitations.

Web Browsers:

Web browsers are mainly used to access websites, but they also help in testing web application security. Choosing the right browser and using extensions can make attacks and analysis easier.

Internet Explorer (IE): IE is widely used and most websites are designed for it, so it works well for testing. It supports ActiveX (unlike other browsers) and has tools to view and modify HTTP requests and responses.

Firefox: Firefox is also popular and supports many useful extensions for testing, like managing proxies, editing requests, and handling cookies. It is important for testing attacks like XSS across different browsers.

Opera: Opera is less used but has useful built-in features like proxy control, cookie editing, and clearing data. It also allows customization and quick access to important tools.

Browser Tips: Browsers can help in testing by using multiple sessions for different users, clearing cookies/cache to start fresh, and opening links in new tabs to explore different parts of an application easily.

Integrated Testing Suites:

Integrated testing suites are advanced tools built around intercepting proxies. They combine multiple tools like proxies, spiders, and scanners to help attackers test web applications more efficiently.

How Tools Work: These tools work with a browser to monitor and analyze all requests and responses. They store data and provide features to test, modify, and explore the application easily.

Intercepting Proxy: This is the most important tool, allowing users to view and modify communication between browser and server. It can even handle secure (HTTPS) traffic by acting as a middleman.

Web Application Spiders: Spiders automatically explore a website by following links and forms. They help map the entire application, including hidden or complex parts like login systems and dynamic content.

Fuzzers and Scanners: These tools automate testing by sending different inputs to find vulnerabilities. They speed up the process and help detect common security issues, though manual testing is still important.

Manual Request Tools: Manual request tools let you send and edit individual HTTP requests and see their responses. They are useful for testing vulnerabilities by making small changes to requests and observing how the application reacts.

Feature Comparison:

Burp Suite: Burp Suite is a powerful and user-friendly tool with advanced features like a smart proxy and a detailed web spider. Its main feature is the Intruder tool, which helps automate custom attacks like testing inputs and finding vulnerabilities.

Paros: Paros is a simpler tool with a basic proxy and spider, but it includes a built-in vulnerability scanner. It can detect common issues like XSS and SQL injection, making it useful for quickly finding easy weaknesses.

WebScarab: WebScarab has a basic proxy and spider, along with a simple fuzzer for testing inputs. It also has a feature to compare responses, helping users see changes when modifying requests.

Vulnerability Scanners

Automated tools scan web apps quickly, mapping content and testing inputs for known flaws. They send crafted requests and analyze responses for vulnerability signatures. Results are reported with evidence, but require human validation and interpretation.

Vulnerabilities Detected by Scanners

Scanners reliably find issues with clear signatures like XSS, SQL injection, and path traversal. They use predefined attack patterns and look for expected responses or errors. However, many variants and complex cases evade detection due to lack of clear signatures.